CS6868 Research Mini-Project

Bounded Lock-Free Queues: From SPSC to MPMC

Nishan Arya
da25m001@smail.iitm.ac.in

Chirag
da25m008@smail.iitm.ac.in

Dikshank Sihag
da25m009@smail.iitm.ac.in

May 1, 2026

Abstract

Lock-free bounded queues are foundational components in high-performance concurrent systems, offering an alternative to allocation-heavy linked-list queues by operating on pre-allocated ring buffers. This project systematically implements and evaluates three bounded queue variants in OCaml 5—Single-Producer/Single-Consumer (SPSC), Multi-Producer/Single-Consumer (MPSC), and Multi-Producer/Multi-Consumer (MPMC)—alongside a partial Disruptor-pipeline port. We additionally implement the classical Michael-Scott unbounded queue and a mutex-protected bounded queue as baselines. All implementations are verified for linearisability using QCheck-Lin and QCheck-STM. Benchmarks across 1–8 threads reveal that: (1) the SPSC queue achieves up to 8.72 M ops/sec with a single producer/consumer pair, delivering competitive throughput without any CAS; (2) the Vyukov MPMC queue and the Disruptor MPSC pipeline both outperform the Michael-Scott queue in their respective regimes, but each degrades sharply under high thread counts; (3) cache-line padding via `Atomic.make_contended` provides a decisive improvement in contended configurations, with a 362% gain observed for the Disruptor MPSC at 4 producers. We answer the central research question: the most dramatic complexity-performance inflection occurs at the MPSC→MPMC boundary, where the benefit of per-slot sequence numbers is eroded by symmetric contention on both producer and consumer cursors.

1 Goals

The lock-free unbounded queues studied in Lecture 08 (the Michael-Scott queue) allocate a fresh heap node for every enqueue operation. This generates continuous pressure on the garbage collector and causes cache misses as nodes traverse the allocator's free lists. Bounded queues backed by pre-allocated ring buffers are the standard remedy in performance-sensitive applications: the allocation is done once at construction time, and thereafter every enqueue/dequeue is a pure array index operation.

The design space of bounded ring-buffer queues spans several points of increasing complexity:

- **SPSC** (single-producer, single-consumer): two monotonically increasing atomic indices; no CAS ever needed.
- **MPSC** (multi-producer, single-consumer): producers race on the tail via CAS; consumer reads head uncontested.
- **MPMC** (multi-producer, multi-consumer): per-slot atomic sequence numbers; both ends use CAS or fetch-and-add (FAA).

The LMAX Disruptor [2] popularised ring-buffer queues in financial trading and introduced the FAA-based multi-producer sequencer with an available-buffer bitmap to enable out-of-order publication. This project builds all three variants in OCaml 5, tests them exhaustively, and benchmarks them against each other and against classical alternatives.

Research question. *How much throughput does a pre-allocated ring buffer gain over the allocation-heavy Michael-Scott queue across different producer/consumer configurations, and where in the SPSC → MPSC → MPMC design space does the complexity-performance trade-off shift most dramatically?*

2 Background

2.1 Ring buffers and monotonic indices

A bounded queue of capacity C (a power of two) is backed by an array `buf[0..C-1]`. Two monotonically increasing integers `head` and `tail` act as logical indices: the slot in use at position p is `buf[p & (C-1)]`. Letting indices grow without wrapping avoids the classic ABA ambiguity between *full* ($\texttt{tail} - \texttt{head} = C$) and *empty* ($\texttt{tail} = \texttt{head}$).

2.2 SPSC: Lamport's queue [1]

With exactly one producer and one consumer, no CAS is ever needed. The producer is the *sole writer* of `tail` and the consumer is the sole writer of `head`. Therefore both can use plain atomic loads and stores. An atomic release-store on `tail` after writing the value ensures the value is visible to the consumer before the tail increment, maintaining the necessary ordering guarantee without a full memory fence.

2.3 MPMC: Vyukov's per-slot sequence numbers [4]

Each slot i carries its own atomic `sequence` field, initialised to i. The lifecycle of a slot at logical position p is:

1. **Free for write**: `sequence = p`. A producer who observes this and wins a CAS on `enqueue_pos` owns the slot.
2. **Written**: producer sets `sequence = p+1`, signalling consumers.
3. **Free for next cycle**: consumer sets `sequence = p + C`, signalling the next producer wave.

This requires *no global lock and no ABA protection*: the sequence number encodes both the slot's generation and its state.

2.4 Disruptor pipeline [2]

The LMAX Disruptor separates concerns differently. A *sequencer* (SPSC or multi-producer) manages a single monotonic cursor. Producers call `next` to claim a sequence number, write to `buf[seq & mask]`, then call `publish`. For multiple producers, `next` uses FAA to claim a batch atomically and then writes a per-slot `available_buffer` flag so consumers can detect out-of-order gaps. Consumers query a *sequence barrier* which scans the available buffer to find the highest contiguous sequence. This design is optimised for fan-out pipelines where *every* consumer processes every event, rather than work-stealing where each event goes to exactly one consumer.

2.5 Michael-Scott queue [3]

The classical lock-free unbounded queue uses a linked list of dynamically allocated nodes. The enqueue operation CAS-swings the tail pointer; dequeue CAS-advances the head. The allocation of one heap node per enqueue is its primary performance liability compared to ring-buffer designs.

3 Tasks Undertaken

3.1 SPSC Queue

Data structure. The SPSC queue is a record with five fields:

Listing 1: Core SPSC record type.

```
type 'a t = {
  capacity : int;
  mask     : int;            (* capacity - 1, for cheap modulo *)
  buf      : 'a Option.t array;
  head     : int Atomic.t;   (* consumer writes; producer reads *)
  tail     : int Atomic.t;   (* producer writes; consumer reads *)
}
```

Both `head` and `tail` are allocated with `Atomic.make_contended`, which instructs the OCaml runtime to place each value on its own cache line. Without this, the two indices share a line and every producer write invalidates the consumer's cached line and vice versa — this is *false sharing*, discussed further in Section 5.4.

try_enqueue. The producer reads `tail` and `head`, checks that `tail` − `head` < `capacity` (the back-pressure condition), writes the value into `buf[tail & mask]`, and then performs a *release store* on `tail`. The release ordering guarantees that the value write is globally visible before the incremented tail becomes visible to the consumer: no explicit fence is required beyond what `Atomic.set` provides in OCaml 5's memory model.

try_dequeue. Symmetrically, the consumer reads `head` and `tail`, returns `None` if `head` ≥ `tail`, reads the value, clears the slot for the GC, and release-stores `head`. No CAS is ever issued: the invariant that only one domain writes each index is the key correctness property.

Listing 2: SPSC enqueue (no CAS required).

```
let try_enqueue t v =
  let tl = Atomic.get t.tail in
  let hd = Atomic.get t.head in
  if tl - hd >= t.capacity then false
  else begin
    t.buf.(tl land t.mask) <- Some v;
    Atomic.set t.tail (tl + 1); (* release store *)
    true
  end
```

3.2 MPMC Queue : Vyukov's Algorithm

Slot structure. Each element of the ring buffer is not just a value but a *slot* with its own sequence number:

Listing 3: Per-slot record for Vyukov MPMC.

```
type 'a slot = {
  sequence : int Atomic.t;
  mutable  value : 'a option;
}
```

The queue-level `enqueue_pos` and `dequeue_pos` are plain `Atomic.make` values (unpadded in the production implementation); the false-sharing investigation in Section 5.4 quantifies the cost of this choice.

Enqueue logic. A producer reads `enqueue_pos = pos`, reads `slot.sequence = seq`, and computes `diff = seq - pos`:

- `diff = 0`: the slot is free for this cycle; the producer CAS-es `enqueue_pos` from `pos` to `pos+1`. If it wins, it writes the value and sets `slot.sequence = pos+1`, signalling consumers.
- `diff < 0`: either the buffer is full (consumer has not yet freed this slot for the current cycle) or a consumer is mid-flight; the producer checks `dequeue_pos` to distinguish the two cases and either spins or returns `false`.
- `diff > 0`: another producer advanced past us; retry.

Dequeue logic. A consumer reads `dequeue_pos = pos`, checks `slot.sequence - (pos+1) = 0` (the slot has been written for this cycle), then CAS-es `dequeue_pos`. On success it reads the value and resets `slot.sequence = pos + mask + 1`, advancing the sequence by one full buffer cycle to free the slot for the next producer wave.

An important subtlety: if `diff < 0` and `enqueue_pos > pos`, a producer has claimed the slot via CAS but has not yet written the value. The consumer must spin rather than return `None`, because the enqueue has already logically committed (the producer claimed the slot and will return `true`). Returning `None` here would violate linearisability. The same reasoning applies symmetrically on the producer side.

3.3 MPSC Queue

The MPSC queue applies Vyukov's per-slot sequence technique but distinguishes the consumer path: because there is exactly one consumer, `dequeue_pos` is advanced with a plain `Atomic.set` rather than a CAS. This eliminates one atomic read-modify-write operation on the hot consumer path.

On the producer side, the spec requires CAS (not FAA) for tail claiming. This is deliberate: FAA irrevocably increments the counter, meaning a producer that claims a slot must always complete the write, even if the buffer is full. CAS can fail and retry without committing, enabling a *non-blocking* `try_enqueue` that returns `false` cleanly when the buffer is full rather than spinning indefinitely.

A subtle addition compared to the pure Vyukov MPMC is a "producer in-flight" detection on the consumer side: if the consumer finds `diff < 0` but observes that `enqueue_pos > pos`, it knows a producer has claimed the slot via CAS and must spin for the value to appear, rather than incorrectly returning `None`.

3.4 Disruptor Pipeline

We implement two pipeline variants:

- **Int_spsc**: wraps `disruptor_sequencer_spsc` + `ring_buffer` + `sequence_barrier`.
- **Int_mpsc**: wraps `disruptor_sequencer_mpsc` (FAA-based cursor with available-buffer bitmap).

SPSC Sequencer. The producer maintains a *cached consumer sequence* to avoid reading the contended consumer atomic on every iteration. Only when the buffer appears full does it refresh the cache by reading the live consumer sequence. This single optimisation eliminates a cross-domain atomic read on the common (non-full) path.

MPSC Sequencer. The multi-producer sequencer uses `fetch_and_add` on the cursor to claim a batch of slots atomically:

Listing 4: FAA-based batch claim in the MPSC Disruptor sequencer.

```
let next t n =
  let hi = Sequence.fetch_and_add t.cursor n + n in
  let lo = hi - n + 1 in
  ...
  (lo, hi)
```

After writing their slots, producers mark each one in `available_buffer` so that out-of-order publications can be detected by the sequence barrier. A TSAN-flagged race on `cached_gating_min` (a mutable field read/written by multiple producer domains) was fixed by changing it to an `Atomic.t`: the cached value is only a performance hint, so any recent valid observation is safe regardless of which producer writes it last.

Why no Disruptor MPMC. FAA is irrevocable: a consumer that increments `dequeue_pos` must complete its read. This makes a non-blocking `try_dequeue` interface impossible without a serialising mutex, which defeats the purpose of FAA-based concurrency. Furthermore, clean shutdown in an `nP:nC` sweep is inherently difficult: a consumer that has claimed a slot via FAA but finds no value published (because the producer was preempted between its FAA and its `available_buffer` write) will spin indefinitely. We therefore test Disruptor only in its natural SPSC and MPSC pipeline topologies.

3.5 Supporting Modules

`sequence.ml` provides a `Sequence.t` — a padded atomic integer (`Atomic.make_contended`) with get, set, cas, and fetch_and_add operations. `sequence_barrier.ml` implements the consumer-side wait: it polls the sequencer's cursor (and, for MPSC, scans the available buffer) to find the highest contiguous sequence that can be safely read. `wait_strategy.ml` provides the `Domain.cpu_relax` spinning strategy used throughout.

4 Testing and Verification

4.1 Smoke Test

A hand-written smoke test (`test_spsc_smoke.ml`) verifies basic enqueue/dequeue ordering, full-buffer rejection, and empty-queue return values for the SPSC queue under sequential execution.

4.2 Stress Tests

Three concurrent stress tests verify that no items are lost or duplicated:

- **SPSC stress**: 1 M items sent from one producer domain to one consumer domain; consumer verifies every item is received exactly once.
- **MPSC stress**: 4 producers × 250 K items; consumer collects all 1 M items and checks the union.
- **MPMC stress**: 4 producers × 4 consumers × 250 K items; a shared counter verifies total items consumed.

All three pass deterministically, confirming no data races on the value path.

4.3 QCheck-STM and QCheck-Lin

The most rigorous testing uses `QCheck-STM` and `QCheck-Lin` from the `multicoretests` library. We model all three queues against the same sequential specification:

Listing 5: Sequential model shared by all three STM/Lin test suites.

```
type state = { contents : int list; capacity : int }
type cmd   = Enqueue of int | Dequeue

let next_state cmd st = match cmd with
  | Enqueue v ->
    if List.length st.contents >= st.capacity then st
    else { st with contents = st.contents @ [v] }
  | Dequeue ->
    (match st.contents with
     | []     -> st
     | _ :: t -> { st with contents = t })
```

Because SPSC is inherently single-producer/single-consumer, direct multi-domain testing by the STM harness would violate the queue's precondition. We wrap each side in a per-side mutex in the test adapter, so the underlying queue code is exercised with correct single-writer invariants while the harness can still exercise interleaved concurrent command sequences.

Three test categories were run for each of the three queue implementations:

1. **STM sequential** (`agree_test`): ensures single-threaded behaviour matches the sequential model.
2. **STM parallel** (`agree_test_par`): generates random parallel command sequences and checks that some sequential interleaving of the observed results is consistent with the model.
3. **Lin linearisability** (`lin_test`): independently verifies linearisability using the QCheck-Lin checker.

All 9 tests (3 queues × 3 test types) pass.

4.4 Comparison Suite

`test_comparison.ml` runs the same STM/Lin battery head-to-head between the spec-correct queues and the Disruptor pipeline variants, plus a deterministic replay test that enqueues a fixed sequence and checks dequeue order. 11 tests pass across SPSC and MPSC design points.

5 Evaluation

5.1 Experimental Setup

All benchmarks were run on a multi-core Linux machine with OCaml 5. Each measurement runs for 2 seconds with a fixed ring buffer of 65 536 slots. Throughput is reported in millions of operations per second (M ops/sec); higher is better. No warm-up period is subtracted: the 2-second window is long enough to dominate start-up noise.

5.2 Thread Sweep

Tables 1 to 4 report throughput across four producer/consumer configurations.

1P:1C baseline (Table 1). With a single producer and consumer, all implementations are on an even footing: there is no contention on either index. The MPSC queue achieves the highest throughput (24.62 M ops/sec), which is counterintuitive at first glance. The reason is that the MPSC queue uses per-slot sequence numbers (cheaper than Option-boxing) and benefits from the fact that its dequeue path needs no CAS. The SPSC queue (6.83 M) is competitive, the Disruptor SPSC (8.56 M) benefits from its cached-consumer optimisation, and the Vyukov MPMC (10.03 M) beats the Michael-Scott queue (6.72 M) despite its extra sequence-number reads. The mutex-protected bounded queue (4.66 M) is the slowest, as expected.

Table 1: 1P:1C throughput (M ops/sec). Single producer, single consumer.

Implementation	M ops/sec
MPSC queue (Vyukov)	24.62
Vyukov MPMC	10.03
Disruptor SPSC	8.56
SPSC queue	6.83
Michael-Scott	6.72
Disruptor MPSC	6.30
Bounded (mutex)	4.66

nP:1C (Table 2). This is the MPSC regime. The Disruptor MPSC dramatically outperforms all others at high thread counts, reaching 12.59 M ops/sec at 6 producers. This is the key advantage of FAA-based claiming: each producer's claim is unconditional, so there is no CAS retry storm. However, the MPSC queue collapses from 25.83 M at 1 producer to 2.00 M at 5+ producers, suffering from CAS contention. The Vyukov MPMC and Michael-Scott queues show similar degradation curves.

Table 2: nP:1C throughput (M ops/sec). n producers, 1 consumer.

Impl \ n	1	2	3	4	5	6	7	8
MPSC queue	25.83	5.23	3.25	2.38	2.00	2.00	1.91	2.00
Disruptor MPSC	7.10	7.40	9.69	10.21	10.10	12.59	12.42	12.54
Vyukov MPMC	7.83	3.57	2.53	2.35	2.16	1.87	1.76	1.22
Michael-Scott	5.96	3.76	2.98	2.47	2.15	2.04	1.79	1.81
Bounded(mutex)	3.88	3.32	2.23	2.04	1.70	1.66	1.41	1.33

1P:nC and nP:nC (Tables 3 and 4). In these configurations only Vyukov MPMC, Michael-Scott, and Bounded(mutex) apply; SPSC and MPSC variants are excluded as they do not support multiple consumers. All three degrade roughly proportionally with thread count: adding a second consumer roughly halves throughput, reflecting the cost of CAS contention on the consumer side. Bounded(mutex) is surprisingly competitive in the 1P:nC case: the single consumer mutex serialises dequeues, but the fixed cost per operation is low. The Vyukov MPMC consistently outperforms Michael-Scott by roughly 1.2×–1.5× across these configurations.

Table 3: 1P:nC throughput (M ops/sec). 1 producer, n consumers.

Impl \ n	1	2	3	4	5	6	7	8
Vyukov MPMC	5.76	3.48	2.92	2.61	2.26	1.54	1.26	1.19
Michael-Scott	6.08	3.02	2.24	1.81	1.42	1.22	1.08	1.02
Bounded(mutex)	4.21	3.30	3.02	2.35	2.07	1.89	1.94	1.63

Table 4: nP:nC throughput (M ops/sec). n producers, n consumers.

Impl \ n	1	2	3	4	5	6	7	8
Vyukov MPMC	6.92	2.97	1.83	1.67	1.84	1.46	1.28	1.21
Michael-Scott	5.74	2.74	1.95	1.62	1.43	1.29	1.22	1.14
Bounded(mutex)	3.87	3.67	2.28	2.19	2.06	1.77	1.69	1.58

5.3 Batch-Size Sweep

Table 5 shows the effect of batch size k on throughput for the 1P:1C configuration.

Table 5: Batch-size sweep — 1P:1C (M ops/sec). Higher k = more items per loop turn.

Impl \ batch k	1	4	16	64	256
Disruptor SPSC	6.09	**32.04**	7.30	7.31	7.71
Vyukov MPMC	6.62	8.17	7.61	8.53	**20.71**
Disruptor MPSC	6.79	6.82	6.37	6.55	6.44
Michael-Scott	5.08	2.74	4.29	5.43	6.34
Bounded(mutex)	4.00	3.62	3.95	3.71	3.69

The Disruptor SPSC shows a dramatic 5× boost at $k = 4$ (32.04 M ops/sec) because the consumer drains all available events per barrier pass, amortising the barrier-scan cost across multiple items. The effect diminishes at $k = 16$ and beyond as the per-item cost re-dominates. The Vyukov MPMC shows a late spike at $k = 256$ (20.71 M), consistent with its CAS-retry cost being amortised over a large batch. The Disruptor MPSC is relatively insensitive to batch size, suggesting its bottleneck is the FAA latency rather than per-item overhead.

5.4 False Sharing

Table 6 quantifies the throughput impact of cache-line padding on atomic indices. Padding is applied via `Atomic.make_contended`; the unpadded baseline uses `Atomic.make`.

Table 6: False-sharing investigation. Delta = (padded − unpadded) / unpadded × 100%.

Implementation	Config	Padded (M/s)	Unpadded (M/s)	Delta
Disruptor SPSC	1P:1C	7.20	7.36	−2.2%
Disruptor MPSC	1P:1C	5.56	4.97	+11.9%
Disruptor MPSC	4P:1C	8.37	1.81	+362.7%
Vyukov MPMC	1P:1C	5.80	7.54	−23.1%
Vyukov MPMC	2P:1C	3.83	2.73	+40.1%
Vyukov MPMC	4P:4C	2.24	1.40	+59.9%

The results reveal two distinct patterns:

Padding helps under contention. The 362.7% gain for Disruptor MPSC at 4P:1C is the headline result. With 4 producer domains all writing to an `available_buffer` indexed near the cursor, every cursor write invalidates the cache line shared by other producers, causing a cache-coherence storm. Padding the cursor sequence to its own line eliminates this entirely.

Padding can hurt in uncontended cases. Vyukov MPMC at 1P:1C shows a 23.1% *reduction* with padding. When only one producer and one consumer access each index, padding spreads them across more cache lines, reducing spatial locality and increasing the number of TLB entries consumed. The breakeven point is reached at 2P:1C (+40.1% improvement with padding), where inter-domain false sharing begins to dominate.

Implications for production code. The SPSC queue uses `Atomic.make_contended` for both `head` and `tail` unconditionally, which is correct: in any realistic deployment the producer and consumer run on different cores, and the −2.2% overhead at 1P:1C is negligible compared to the protection afforded. The Vyukov MPMC production code uses `Atomic.make` (unpadded) for its queue-level indices, which is suboptimal at higher thread counts; `make_contended` should be used for any deployment with 2+ producers or 2+ consumers.

6 Reflection on the Use of LLMs

Tools and models used. We used Claude (Sonnet 4.6) for code generation assistance, debugging, and report drafting. GitHub Copilot was used inline within the editor for boilerplate completion. ChatGPT (GPT-4o) was consulted for explaining certain aspects of the OCaml 5 memory model and for cross-checking our understanding of the Vyukov queue algorithm.

What worked well. Claude was highly effective at generating the QCheck-STM adapter boilerplate, which is structurally repetitive across the three queue variants. Given a description of the sequential model and the queue interface, it produced a correct `module SpscAdapter` on the first attempt, including the subtle choice to wrap each side in a per-side mutex for the test harness without modifying the underlying queue. It also correctly explained the slot-lifecycle invariants in Vyukov's algorithm when we described the `diff` cases in informal terms.

What did not work. The LLM initially generated an incorrect `available_buffer` indexing formula for the MPSC Disruptor sequencer (using `pos mod size` instead of `(pos lsr index_shift) land mask`), which produced a subtle bug where out-of-order detection failed for large sequence numbers. TSAN did not catch this because it is a logical error, not a data race; we caught it by running the STM parallel tests which found a failing linearisability trace.

What was surprising. Both positively and negatively: the LLM was better than expected at explaining *why* the sequence-barrier spin is necessary (the "producer in-flight" case), but significantly worse than expected at reasoning about the precise memory ordering required. When asked "does this need a fence?" it consistently answered with hedges like "it depends on the architecture" rather than reasoning from OCaml 5's defined memory model.

What was difficult. Justifying the linearisation points for the MPSC dequeue "producer in-flight" spin required genuine understanding of what linearisability means: the linearisation point of a successful `try_enqueue` is the CAS on `enqueue_pos`, not the `Atomic.set slot.sequence` that publishes the value. Once the CAS commits, the enqueue is logically complete; any subsequent `try_dequeue` at that slot must eventually return the value, not `None`. The LLM could not reason about this precisely and kept conflating "the value is visible" with "the operation is complete." This was the part of the project we had to reason through entirely ourselves.

Overall assessment. The LLM net-helped on scaffolding, boilerplate, and initial drafts, but was unreliable on the subtle concurrent-correctness reasoning that constitutes the core intellectual contribution of the project. We would use it similarly again, but with more scepticism on anything touching memory ordering or linearisation arguments.

7 Conclusions

Answering the research question. A pre-allocated ring buffer gains substantial throughput over the Michael-Scott queue in almost every configuration: 1.3×–3.7× at 1P:1C depending on the specific variant. The advantage is largest in the SPSC and MPSC regimes where per-slot contention is absent or one-sided. The advantage narrows in the MPMC regime: at 4P:4C, the Vyukov MPMC leads the Michael-Scott queue by only ∼1.03× (1.67 vs 1.62 M ops/sec), because the CAS contention on both ends largely erases the allocation savings.

The MPSC→MPMC inflection. The most dramatic complexity-performance inflection occurs precisely at the MPSC→MPMC boundary. Moving from MPSC (CAS on tail only) to MPMC (CAS on both tail and head) doubles the CAS burden under load. Meanwhile the Disruptor's FAA-based MPSC grows with more producers rather than shrinking, reaching 12.59 M ops/sec at 6 producers — a fundamentally different scaling curve that arises because FAA does not involve a retry loop.

False sharing is the dominant micro-architectural effect. The 362% throughput gain from padding the Disruptor MPSC cursor is the single most surprising result. It confirms that for shared-memory concurrent data structures on multi-core CPUs, cache-line discipline is not an optional optimisation but a correctness-adjacent concern: an unpadded implementation can be 4× slower than the padded one under realistic thread counts.

Limitations and future work. Our benchmarks were conducted on a single machine; results on machines with NUMA topology or different cache-line sizes (e.g., Apple Silicon) may differ. The Disruptor MPMC was not implemented due to the FAA irrevocability problem; a future implementation could use a hybrid approach (FAA with a cancellation protocol) to recover non-blocking semantics. It would also be valuable to study the queues under realistic workloads with non-trivial item-processing costs, where the relative weight of queue overhead vs. application work changes the optimal design point.

Contributions

Member	Contribution (%)	Main responsibilities
Nishan	33%	SPSC, MPSC implementations; QCheck-Lin/STM tests; stress tests
Chirag	34%	MPMC and Disruptor pipeline; false-sharing benchmark, presentation
Dikshank	33%	Thread/batch sweep benchmarks; evaluation analysis; report
Total	**100%**	

References

[1] Leslie Lamport. Specifying concurrent program modules. *ACM Transactions on Programming Languages and Systems*, 5(2):190–222, 1983.

[2] LMAX Group. The LMAX disruptor. https://lmax-exchange.github.io/disruptor/, 2011. Accessed 2026.

[3] Maged M. Michael and Michael L. Scott. Simple, fast, and practical non-blocking and blocking concurrent queue algorithms. In *Proceedings of the 15th ACM Symposium on Principles of Distributed Computing (PODC)*, pages 267–275, 1996.

[4] Dmitry Vyukov. Bounded MPMC queue. https://www.1024cores.net/home/lock-free-algorithms/queues/bounded-mpmc-queue, 2010. Accessed 2026.